Projeto de Engenharia Informática

# Linguagens Naturais Controladas (LNC)

## O Design de Português Determinístico para Autorização Corporativa e Controlo de Acesso.

Uma Síntese de Investigação Lógica e Arquitetura Computacional.

# A Armadilha da Ambiguidade

## Linguagem Informal (Ambígua)

*“Ele pode alterar isso se tiver as permissões certas.”*

- Ele = Quem?
- Isso = O quê?
- Pode = Capaz ou autorizado?

## LNC (Controlada)

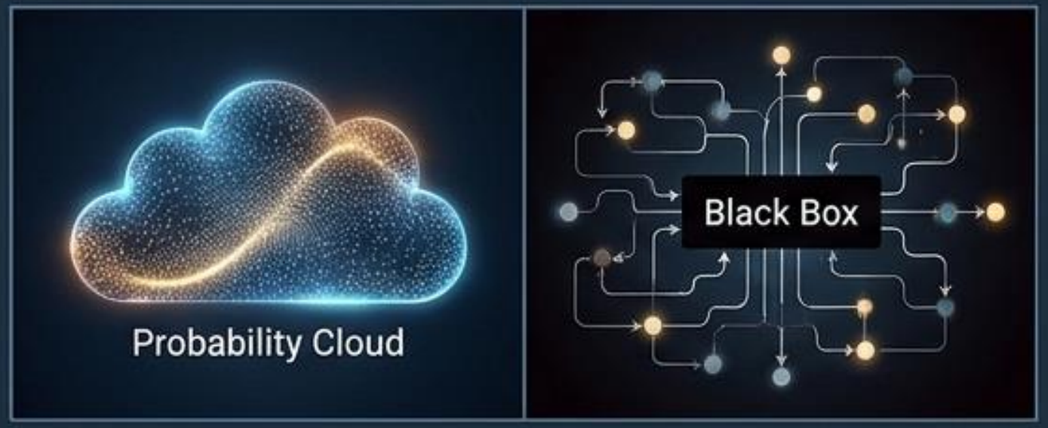
*“O utilizador pode alterar o pedido se possuir permissão de edição.”*

Sujeito, objeto e condição são explícitos. Uma única leitura possível para a máquina.

# O Risco de Alucinação em LLMs

Modelos de Linguagem de Grande Escala (**LLMs**) dependem de inferência **probabilística**. Em políticas de segurança corporativa, a probabilidade **não é suficiente**.

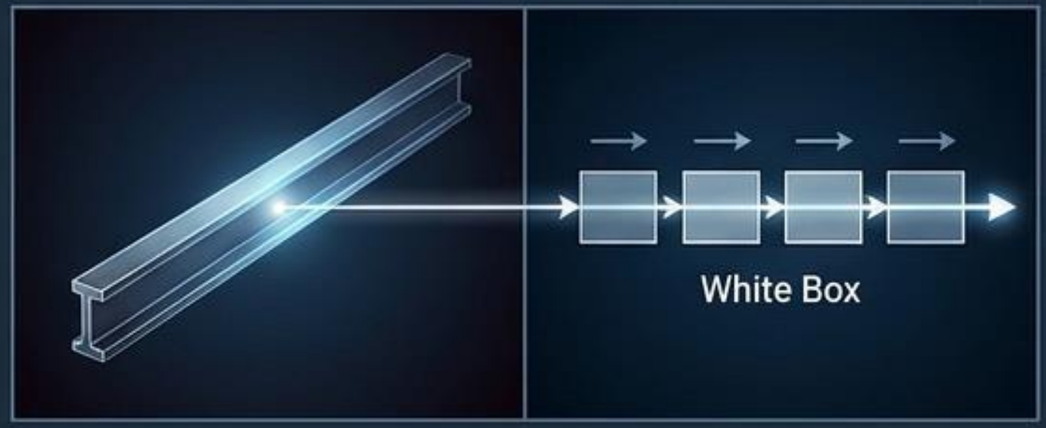
## Abordagem **LLM**




**Mecanismo:** Tradução direta (Texto Livre -> LLM -> Código).

**Falha:** Risco de **não-determinismo**, **alucinações semânticas** e interpretações variáveis.

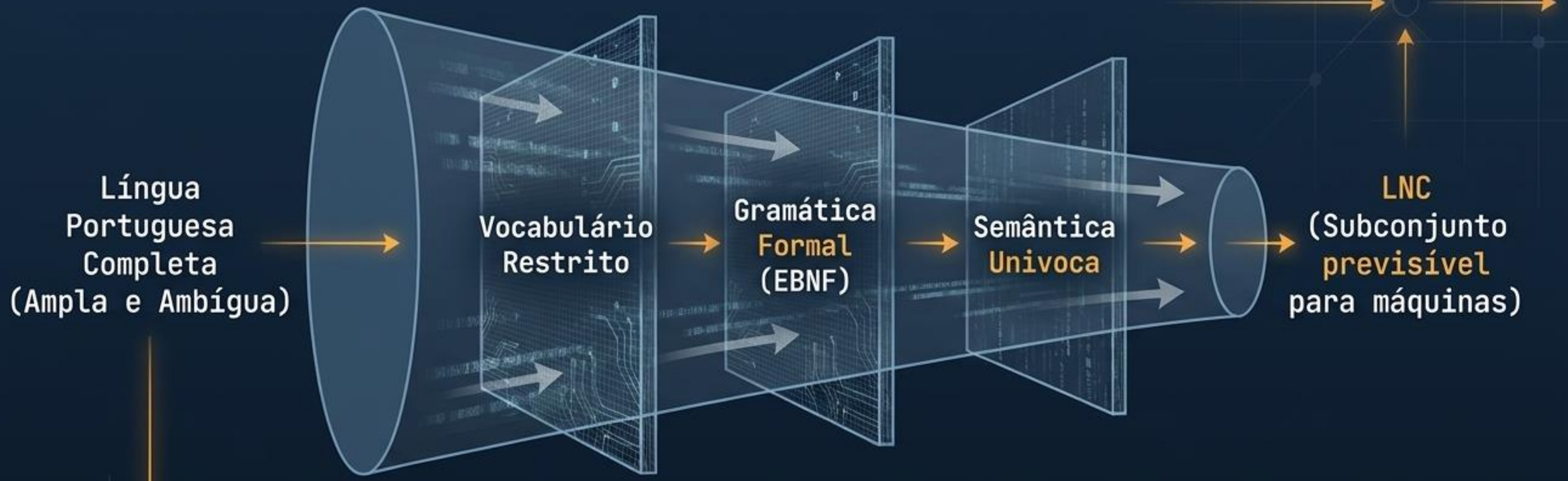
## A Necessidade **LNC**




**Mecanismo:** Representação Intermédia (Texto Livre -> LNC -> Execução).

**Vantagem:** 100% de precisão. O LNC serve como ponte **determinística**, **eliminando ambiguidades** antes do processamento lógico.

# O que é uma Linguagem Natural Controlada?




**Objetivo Final:** Preservar a legibilidade humana enquanto se converte o texto num formato que uma máquina consiga processar como código.

# Orientação: Humanos vs. Máquinas

## HOCL (Human-Oriented)

**Foco**: Legibilidade e consistência técnica.

**Objetivo**: Facilitar a comunicação para falantes não-nativos.

**Exemplos:** Manuais de aeronáutica (ASD-STE100).

## MOCL (Machine-Oriented)

**Foco**: **Eliminação de ambiguidade** e tradução para **lógica**.

**Objetivo**: Processamento por **parsers** para **representação de conhecimento**.

**Exemplos:** Attempto Controlled English, Grammatical Framework.

**O Nosso Foco (MOCL):** Desenvolver um fragmento de Português onde as **regras são estritas** o suficiente para **validação determinística** por um **compilador**.

# O Espaço de Classificação PENS (Kuhn)

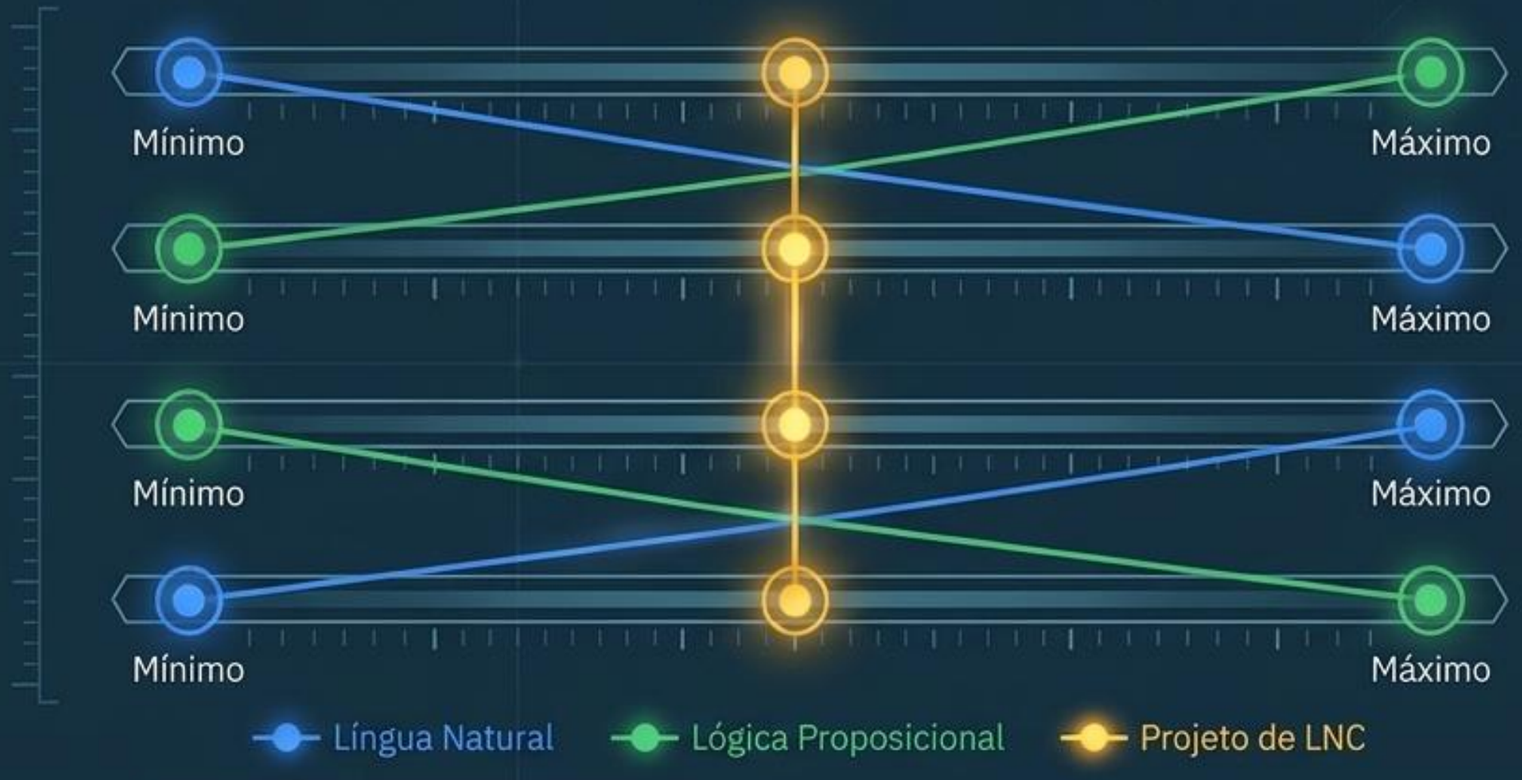
Posicionando a LNC entre o Caos e o Código.

**P**recisão
(Eliminação de Ambiguidade)

**E**xpressividade
(Capacidade de representar conceitos complexos)

**N**aturalidade
(Leitura intuitiva)

**S**implicidade
(Facilidade de gramática e estrutura)




A nossa LNC sacrifica intencionalmente a **Expressividade** para maximizar a **Precisão**, mantendo uma **Naturalidade** suficiente para o domínio corporativo.

# A Lógica do Sistema: Regras de Controlo

## Regras Proscritivas (O que evitar)

- Sem pronomes vagos: `'Ele', 'Isso', 'Aquele'` não podem ser resolvidos deterministicamente.
- Sem sinónimos: Um conceito = Uma palavra.
- Sem subordinações longas: Bloqueio de frases compostas complexas.

## Regras Prescritivas (Padrões Obrigatórios)

- Estrutura Rígida: `[Sujeito] + [Verbo] + [Objeto] + [se Condição]`
- Vocabulário Fixo: Utilização estrita de dicionários de domínio pré-aprovados.
- Condições Explícitas: Todas as dependências devem ser declaradas nominalmente.

# Domínio Alvo: Políticas de Controlo de Acesso

O fragmento de Português foi otimizado para expressar relações deônticas em ambientes corporativos.




Ao focar nestas restrições corporativas, a LNC garante uma verificação de segurança imutável e auditável.

# Domínio Alvo II: Auditoria e Monitorização de Risco

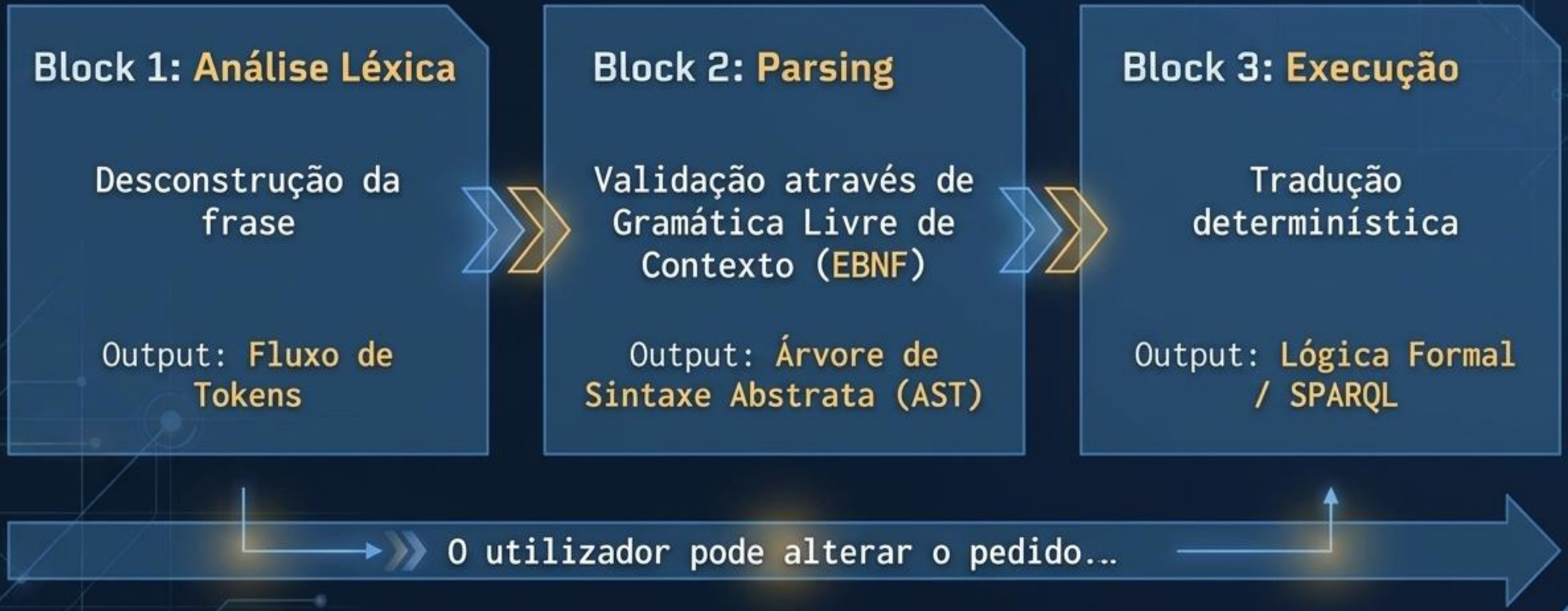
O fragmento de Português foi otimizado para expressar regras de risco e consultas analíticas no grande retalho.




Ao focar nestas métricas operacionais, a LNC garante uma auditoria de risco imutável e em tempo real.

# A Arquitetura de Processamento

Como a máquina interpreta a nossa CNL (Pipeline em Python).



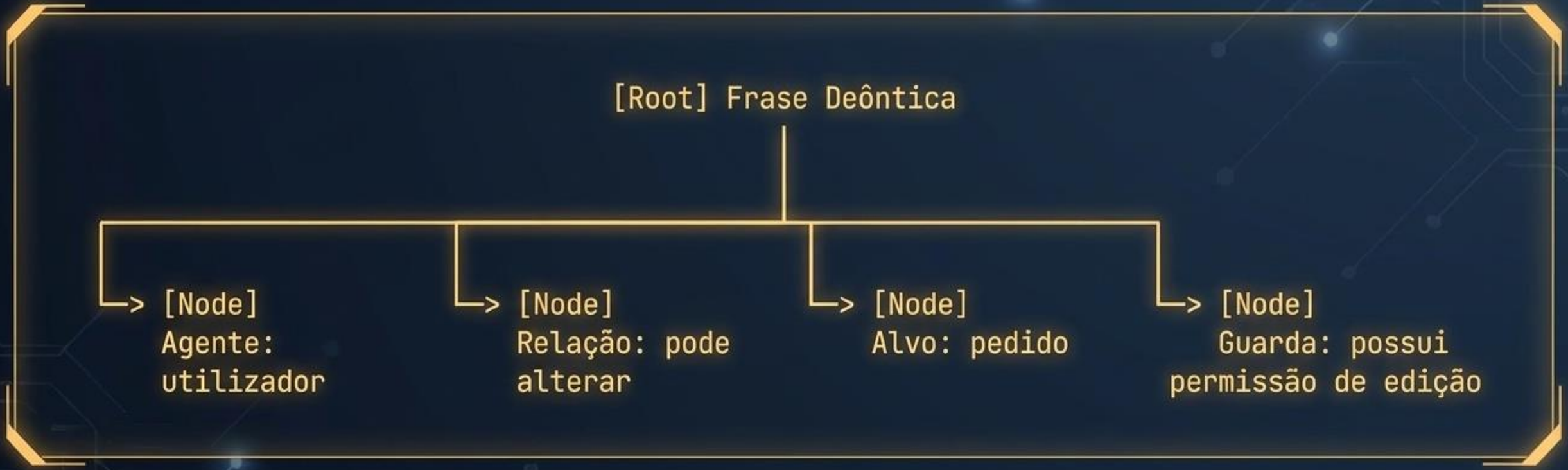
# Passo 1: Análise Léxica e Dicionários Controlados

O Lexer segmenta a entrada e classifica cada elemento através de um dicionário rigoroso, distinguindo palavras funcionais de termos de domínio.



# Passo 2: Parsing Determinístico e Geração de AST

O Parser aplica as regras EBNF (Extended Backus-Naur Form) aos tokens. Se a gramática for respeitada, gera uma Árvore de Sintaxe Abstrata (AST).

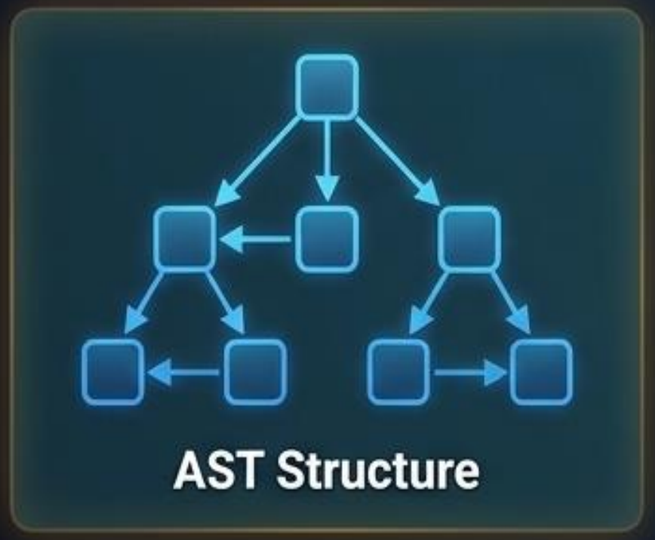



**Key Insight**

A AST isola as entidades e relações lógicas, criando uma representação puramente estrutural, livre de qualquer vestígio de linguagem natural.

# Passo 3: Execução e Lógica Formal

A AST é consumida por um motor de regras, convertendo a estrutura intermédia em consultas acionáveis (ex: SPARQL) ou provas lógicas.



AST Structure

compiler

```
possui(U, edição) ->
  pode_alterar(U, pedido)

SELECT ?user WHERE {
  ?user ex:hasPermission
ex:edit ...
}
```

**Academic Context**

Como demonstrado pela pesquisa de Luz e Caio et al., esta representação intermédia é a chave para interagir com Grafos de Conhecimento (Knowledge Graphs) de forma 100% precisa.

# O Novo Paradigma: O Melhor de Dois Mundos

| | Linguagem Natural Pura (LLM) | Código Puro (Python/SPARQL) | LNC (O Nosso Projeto) |
|---|---|---|---|
| Legibilidade Humana | ● Alto | ○ Baixo | ● Alto |
| **Precisão Determinística** | ○ Baixo/Não-Determinístico | ● Alto | ● Alto |
| **Baixa Barreira de Entrada** | ● Alto | ○ Baixo | ◑ Médio |

**Synthesis Note:** O **LNC** funciona como o **'Missing Link'**. Modelos recentes provam que treinar/utilizar LLMs para gerar LNC (em vez de código complexo como SPARQL) **reduz drasticamente** as necessidades de dados e erros de sintaxe.

# Conclusão: O Compromisso Essencial

| | |
|---|---|
| **Menos Ambiguidade** | Ao restringir o vocabulário e a gramática, garantimos que cada frase corporativa tenha apenas uma interpretação possível. |
| **Previsibilidade Total para Máquinas** | O pipeline léxico e o parser convertem texto numa Árvore de Sintaxe Abstrata (AST) e em lógica matemática perfeitamente verificável. |
| **O Futuro (Sinergia LNC + RAG/LLM)** | O compromisso compensa: Trocamos expressividade literária por segurança sistémica. No futuro, a integração com RAG (Retrieval-Augmented Generation) permitirá a geração de consultas em ambientes sem margem para erro. |

**A precisão não é uma limitação da linguagem;**
**é uma funcionalidade da arquitetura.**